

ObserveOPS AI Chat — Designer's Version

Everything explained in design language. No engineering words.

1. What am I actually designing?

One chat panel. Not three. It has a mode switcher at the top with three options.

Think of it like a text editor that has "Read", "Edit", and "Publish" modes. Same window, different powers.

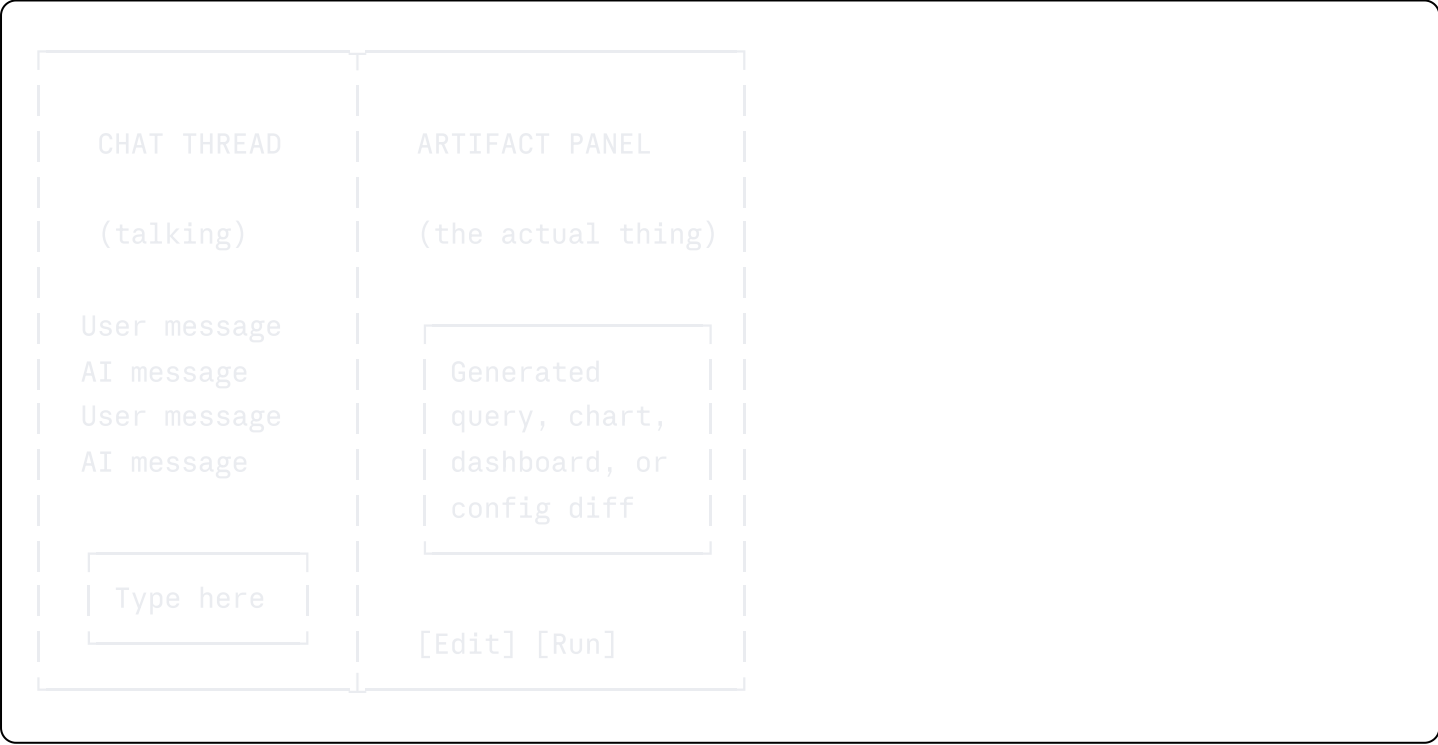
Mode	Simple meaning	Risk
Normal	It talks. Gives answers.	Safe
Query	It writes a search for you. You check it, then run it.	Medium
Workflow	It creates real things (dashboards, alerts, configs).	Dangerous

The risk level is the whole reason the UI changes between modes. Normal mode just shows text. Workflow mode needs a big "Approve" button before anything happens.

2. The layout (draw this first)

The most important rule

Split into two columns.



Left = conversation. Short messages. "Here's what I found." **Right = the deliverable.** The query, the chart, the dashboard preview.

Why: if you put a big query block inside a chat bubble, the user scrolls up and down forever and loses it. Keep the thing they care about pinned on the right. This is exactly how Claude Artifacts and Cursor work.

Four places the chat can live

Where	Size	When
Side panel (default)	400px wide, drag to resize 360-560px	User is working in a module and wants help
Full page	Two columns, 1200px max	Deep investigation or building a dashboard
Command palette	Centered popup, 640px	Quick question, keyboard user
Inline	Small, inside a card	"Explain this alert" button on an alert card

Responsive: under 1024px the side panel becomes an overlay. Under 640px it goes full screen.

3. The context bar (your secret weapon)

Put a row of small removable chips just above the input box:



This shows the user **what the AI can currently see**. Time range, selected service, active filters.

Why this matters so much: without it, users feel like they're talking to a black box. They don't know if the AI knows which service they selected. With it, they can see and fix it in one click.

This one component solves more trust problems than anything else you will design.

4. The three modes as screens

Mode 1 — Normal Chat

What the user sees: a normal chat. Question in, answer out.

Message structure — always in this order:

- 1. The answer (1-2 lines, bold or first)
- 2. The evidence (small text, bullet points)
- 3. [Show reasoning ►] — collapsed by default
- 4. Source links (small chips at bottom)
- 5. Follow-up chips

Example message to put in your Figma mockup:

p95 latency on `checkout` rose from 240ms to 1.8s between 13:52–14:10. Most likely cause: the `payments-api` dependency — its error rate hit 12% in the same window. [1]

3 correlated signals [Show reasoning ▶]

Sources: [1] APM service map [2] payments-api logs

[Show payments-api errors] [Compare to yesterday] [Draft summary]

Components you need: message bubble, citation chip, citation popover on hover, follow-up chip, thumbs up/down, copy button, collapsed reasoning accordion.

Mode 2 — Query Chat

What the user sees: they type in English, the AI writes the search query, **and shows it to them before running it.**

The one rule you must never break: never hide the query.

Here's why, in plain terms. When you ask an AI "show me errors", it doesn't know if you mean HTTP 500 errors or application crash logs. But it will still confidently write a query. It will look correct. It will run. It will give you the wrong numbers, and you'll never know.

Research on this found roughly **1 in 5 real user questions are ambiguous like this.** Even the best models get these wrong about 30% of the time.

You cannot fix this with a better model. **You fix it with UI.** So:

The query block component:

```
{service="checkout"} | status >= 500
| stats count by host
| sort -count
```

← syntax highlighted

```
In plain English:
Filters checkout logs to 5xx errors in
the last hour, counts by host.
```

← one sentence

```
Resolved: service=checkout · last 1h
```

← what it matched

```
[Edit] [Run] [Open in query editor]
```

Four parts: **the query, plain English explanation, what it resolved, actions.**

When AI can't figure something out, it must ask instead of guessing:

I found two things called "errors". Which one do you mean? [HTTP 5xx responses]

[Application exceptions]

Components you need: query block, syntax highlight styles, explanation panel, resolved-context row, result table (with skeleton state), result chart, clarification chips, error card with

suggested fix.

Mode 3 — Workflow Chat

What the user sees: the AI proposes a plan, shows a preview, and waits for a button click before doing anything real.

The flow — four screens:

1. PLAN	2. PREVIEW	3. APPROVE	4. DONE
"I'll do these 3 steps"	Show the diff of what changes	Big button + warning if risky	Success + Undo button

The approval card (design this carefully):

Plan 1. Create metric monitor on checkout 2. Condition: error rate > 5% for 5m 3. Notify: #checkout-oncall [View full config diff →] ▲ This creates a new alerting rule in Production. [Approve & create] [Edit] [Cancel]
--

How much friction to add — use this grid:

	Can undo	Cannot undo
Big impact	Show approval, allow undo after	Hard stop. Approval + type-to-confirm
Small impact	Just do it, show what happened	Do it, log it

Examples: creating a personal saved search = just do it. Pushing a config to 12 production switches (NCCM) = hard stop, show the diff, require typing.

One important detail: make the "Approve" button and the "Edit" button visually **equal weight**. Do not make Approve prettier or bigger. Salesforce discovered that if the accept button looks more attractive, people click it without reading. Same color, same size, same font.

Components you need: plan card with numbered steps, diff view (added/removed lines), approval gate card, warning banner, progress panel, success card with undo button, permission-

denied card.

5. States you must design (the boring but essential part)

Designers forget these. They're 70% of the real work.

State	What it looks like	Text to write
Empty / first open	Scope sentence + 3-6 prompt chips	"Ask about your logs, metrics, alerts, and configs. I can see: APM · checkout · last 1h"
Thinking	Pulsing avatar + text label	"Thinking..." → "Searching logs..." → "Resolving entities..."
Streaming	Text appearing + blinking caret	—
Tool running	Small collapsible chip	"Ran query · 1,204 rows · 320ms"
Empty result	Not an error! Different message	"Query ran — no matching data in the last hour. [Widen to 24h] "
Query failed	Show error + the query + a fix	"Query failed: unknown field hostname . Did you mean host ?"
No permission	Card, not a scary error	"You don't have permission to modify production monitors. I've saved a draft. [Request approval] "
Partial failure	Amber, show what worked	"Steps 1-2 done. Step 3 failed: timeout. [Retry step 3] [Roll back] "
User pressed stop	Freeze text, offer options	"Stopped. [Continue] [Edit prompt] "
Out of scope	Friendly redirect	"I focus on your ObserveOPS data. I can help with..."

Golden rule: never a dead end. Every error must offer a next step or a manual path.

6. Micro-interactions (timing values you can put in Figma/Smart Animate)

Keep it calm. This is an enterprise dark UI, not a consumer app.

Thing	Duration	Easing
Streaming caret blink	1000ms loop	steps
Skeleton shimmer	1200–1500ms	ease-in-out
Thinking pulse	1400ms, opacity 0.5→1	ease-in-out
Tool chip expand	180ms	ease-out
Citation popover	120ms fade + 8px rise, 300ms delay before showing	ease-out
Diff lines reveal	220ms, 20ms stagger per line	ease-out
Approval card appear	200ms, scale 0.98→1	ease-out

Always add a reduced-motion version: replace everything with a 100ms fade.

7. Design system additions

Colors to add to your tokens

- `ai/accent` — one violet/indigo. **Only** for AI-generated content. Never for normal UI.
- `ai/accent-subtle` — background tint version
- `ai/border`
- `confidence/high` `confidence/medium` `confidence/low`
- `diff/added-bg` `diff/added-fg` `diff/removed-bg` `diff/removed-fg`

Check contrast on your dark background. Violet is the color most likely to fail 4.5:1.

The "AI signature"

Every AI message gets a small mark — a sparkle icon + accent color. Nothing else in the product uses it. This is how users instantly know "a machine wrote this, I should check it." IBM, Salesforce, and Shopify all do exactly this.

Type

- `chat/body` 14–15px
- `chat/mono` for queries — IBM Plex Mono fits your existing direction
- `chat/caption` for citations and metadata

Rules for every state

- Never use color alone.** Diffs need `⊕` / `⊖` icons. Confidence needs a word, not just green/amber/red.
- Minimum tap target 24×24px** for chips, thumbs, and toggles (WCAG 2.2 requirement).
- Show confidence as words — "High confidence", "Uncertain" — never as "0.87".

8. Figma components to build

Build these as auto-layout with variants. Start with the bold ones.

Component	Variants
ChatPanel	side / overlay / full-page
MessageBubble	user / ai / system × streaming / complete / error
Composer	default / disabled / streaming (shows Stop)
PromptChip	global / contextual / historical
ContextChip	time / entity / filter, removable
QueryBlock	editable / read-only / running / error
ApprovalGate	standard / destructive / permission-denied
DiffView	query / config / dashboard
ToolCallCard	collapsed / expanded / running / failed
CitationChip + Popover	doc / log / dashboard / metric
ThinkingIndicator	dots / step-list
ConfidenceBadge	high / medium / low
ResultTable	default / skeleton / empty
PlanCard	draft / approved
ErrorCard	permission / query / partial / capacity
EmptyState	first-run / cleared / no-results
ModeSwitcher	normal / query / workflow
FeedbackControls	default / submitted
HistoryItem	default / pinned

Use shared properties named `state`, `mode`, `confidence`, and `severity` across components so everything stays consistent.

9. Writing style for all AI text

The AI should sound like a **calm senior SRE colleague**. Not a friendly assistant.

Do:

- Answer in the first line
- Short sentences
- Say "I'm not sure" when unsure
- Say exactly what will change before doing it

Don't:

- "Great question!"
- Emoji
- Exclamation marks
- Long paragraphs before the answer
- Invent hostnames or service names

Research from Nielsen Norman Group: users want **quick answers, not conversation**. Give the short answer, then offer follow-up chips so they can pull more if they want.

10. What to design first

Week 1-2 — Foundation The shell only. ChatPanel, MessageBubble, Composer, ContextBar, plus every state from Section 5. Get empty / thinking / streaming / error right before anything else. Everything reuses these.

Week 3-4 — Normal Chat Only in Alerts, APM, Logs. Read-only, so no risk. This is where you tune your tone and citation style.

Week 5-7 — Query Chat The query block, explanation, resolved context, edit-and-rerun, and all the query error states.

Week 8+ — Workflow Chat Start with safe reversible actions only (personal dashboard, saved search). Then monitors and alerts with approval gates. NCCM config push last — it's the most dangerous.

Later: proactive suggestions, voice, chat history and search, memory panel, admin dashboard.

Quick reference — if you remember only five things

1. **Two columns.** Chat left, artifact right.
2. **Always show the query** before running it, with a plain-English explanation.
3. **Approval gate** before anything that changes real data. Approve and Edit buttons get equal visual weight.
4. **Context chips** above the input so users see what the AI knows.
5. **Every error offers a next step.** Never a dead end.